Lesson 62: Cooperative Multi-Agent Reinforcement Learning

AI/ML Mathematics

Contents

1 Motivation and Overview **2**

2 References **3**

3 Decentralized Partially Observable MDP **3**

4 Centralized Training with Decentralized Execution **5**

5 Value Decomposition Networks (VDN) **6**

6 QMIX and the IGM Condition **7**

7 QTRAN: Beyond Monotonic Decomposition **9**

8 Multi-Agent Proximal Policy Optimization **10**

9 Credit Assignment **11**

9.1 Difference Rewards 12

9.2 Shapley Value-Based Credit Assignment 12

9.3 Counterfactual Baselines: COMA 12

10 Exercises **13**

11 Summary **14**

12 Further Reading **15**

1 Motivation and Overview

Consider a team of N warehouse robots operating in a shared facility. Each robot must pick up packages from storage shelves and deliver them to shipping docks. The team receives a single shared reward: the total number of packages delivered per hour. Each robot observes only its local surroundings—nearby shelves, packages in its gripper, and robots within sensor range—yet must coordinate with the rest of the team to avoid collisions, balance workload across docks, and maximize throughput. A centralized controller that observes the entire warehouse could, in principle, compute an optimal joint policy; but communication latency and bandwidth constraints require each robot to act based on local observations alone.

This tension between *global cooperation* and *local execution* lies at the heart of cooperative multi-agent reinforcement learning (MARL). Formalizing the problem leads to the Decentralized Partially Observable Markov Decision Process (Dec-POMDP), a model so general that finding an optimal joint policy is NEXP-complete. To make the problem tractable, practical algorithms adopt the Centralized Training with Decentralized Execution (CTDE) paradigm: during training, a centralized process has access to global state and joint actions; during execution, each agent acts using only its local observation history. The tools developed in this lesson will let us formalize the Dec-POMDP model, understand its computational barriers, and derive the value decomposition and policy gradient methods that make cooperative MARL feasible.

Why Cooperative MARL Matters

- **StarCraft micromanagement (SMAC).** Teams of units must coordinate attack focus and positioning using only local observations; QMIX and MAPPO achieve state-of-the-art performance on this benchmark [Samvelyan et al., 2019].
- **Autonomous vehicle platooning.** Vehicles in a platoon share a common fuel-efficiency objective but each vehicle controls only its own throttle and braking based on local sensor readings.
- **Robotic manipulation teams.** Multiple robotic arms assembling a product must coordinate grasps and placements; credit assignment determines which arm's action contributed to success or failure.
- **Network routing.** Routers in a communication network cooperatively minimize global latency while each router observes only local queue lengths.

Prerequisites. Stochastic games and Markov perfect equilibrium (Lesson 61, Definitions 3.1–3.3). MDP formulation, Bellman equations, and value iteration (Lessons 39–42). Policy gradient theorem and actor-critic methods (Lessons 54–57). PPO clipped surrogate objective (Lesson 57, Section 4 in Phase 04). Nash equilibrium and minimax theorem (Lesson 20, Phase 01 game theory).

Learning objectives. After completing this lesson, you will be able to:

- **Define** the Dec-POMDP model and **explain** how it specializes the stochastic game framework to the cooperative setting.
- **State** and **explain** the NEXP-completeness result for optimal Dec-POMDP policy computation.
- **Derive** the VDN additive value decomposition and **prove** that greedy action selection decomposes under it.
- **State** the Individual-Global-Max (IGM) condition and **prove** that the QMIX monotonicity condition implies IGM.
- **Analyze** the representational hierarchy VDN $\subset$ QMIX $\subset$ QTRAN and **construct** examples demonstrating strict inclusions.

- **Describe** the MAPPO algorithm and **explain** how centralized advantage estimation supports decentralized policy improvement.

2 References

- Bernstein et al. [2002] – Original proof of NEXP-completeness for Dec-POMDPs; the foundational complexity result of Section 3.
- Oliehoek and Amato [2016, Chapters 2–5] – Comprehensive textbook treatment of Dec-POMDPs, including exact and approximate solution methods.
- Rashid et al. [2018] – Introduces QMIX and the monotonicity condition for value factorization; primary reference for Section 6.
- Zhang et al. [2021, Sections 4–5] – Survey of cooperative MARL algorithms covering value decomposition and policy gradient methods.
- Yu et al. [2022] – Empirical and theoretical analysis of MAPPO; demonstrates the surprising effectiveness of centralized-critic PPO in cooperative games.

3 Decentralized Partially Observable MDP

The cooperative multi-agent problem is formalized by the Dec-POMDP, which extends the single-agent POMDP to N agents sharing a common reward.

Definition 3.1 (Decentralized POMDP)**.** A *Decentralized Partially Observable Markov Decision Process* (Dec-POMDP) is a tuple

$$\mathcal{M} = (\mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i\in\mathcal{N}}, \mathcal{P}, R, \{\Omega_i\}_{i\in\mathcal{N}}, O, \gamma),$$

where:

- $\mathcal{N} = \{1, \dots, N\}$ is the finite set of agents.
- $\mathcal{S}$ is a finite set of global states.
- $\mathcal{A}_i$ is the finite action set of agent i; the joint action space is $\boldsymbol{\mathcal{A}} = \mathcal{A}_1 \times \cdots \times \mathcal{A}_N$.
- $\mathcal{P} : \mathcal{S} \times \boldsymbol{\mathcal{A}} \to \Delta(\mathcal{S})$ is the state transition function, where $\mathcal{P}(s' \mid s, \mathbf{a})$ is the probability of transitioning to s' given state s and joint action $\mathbf{a} = (a_1, \dots, a_N)$.
- $R : \mathcal{S} \times \boldsymbol{\mathcal{A}} \to \mathbb{R}$ is the shared (team) reward function.
- Ω_i is the finite observation set of agent i, with joint observation space $\boldsymbol{\Omega} = \Omega_1 \times \cdots \times \Omega_N$.
- $O : \mathcal{S} \times \boldsymbol{\mathcal{A}} \to \Delta(\boldsymbol{\Omega})$ is the observation function, where $O(\mathbf{o} \mid s', \mathbf{a})$ gives the probability of joint observation $\mathbf{o} = (o_1, \dots, o_N)$ after transitioning to s' via action $\mathbf{a}$.
- $\gamma \in [0, 1)$ is the discount factor.

Remark 3.2. A Dec-POMDP is a special case of a partially observable stochastic game (POSG, Lesson 61) in which all agents share the same reward function R. This common-payoff structure makes the problem *cooperative*: the agents' interests are perfectly aligned. When $N = 1$, the Dec-POMDP reduces to a standard POMDP; when observations fully reveal the state ($\Omega_i = \mathcal{S}$ and O is the identity), it reduces to a multi-agent MDP.

Observation histories and local policies. Since agent i cannot observe the global state, it must base decisions on its *observation-action history*. At time t, agent i has observed the sequence

$$\tau_i^t = (o_i^0, a_i^0, o_i^1, a_i^1, \dots, o_i^{t-1}, a_i^{t-1}, o_i^t) \in (\Omega_i \times \mathcal{A}_i)^t \times \Omega_i.$$

We write $\mathcal{T}_i^t$ for the set of all such histories of length t, and $\mathcal{T}_i = \bigcup_{t=0}^{\infty} \mathcal{T}_i^t$ for the set of all finite histories.

Definition 3.3 (Local and Joint Policies)**.** A *local (decentralized) policy* for agent i is a mapping

$$\pi_i : \mathcal{T}_i \to \Delta(\mathcal{A}_i),$$

which assigns a distribution over actions to each observation-action history. A *joint policy* is the tuple $\boldsymbol{\pi} = (\pi_1, \ldots, \pi_N)$.

Definition 3.4 (Joint Value Function)**.** Given a joint policy $\boldsymbol{\pi}$ and initial state $s_0 \in \mathcal{S}$, the *joint value function* is

$$V^{\boldsymbol{\pi}}(s_0) = \mathbb{E}^{\boldsymbol{\pi}}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \mathbf{a}_t) \,\middle|\, s_0\right],$$

where the expectation is over the stochastic process induced by the transition kernel $\mathcal{P}$, observation function O, and local policies π_i.

The optimization problem is to find a joint policy $\boldsymbol{\pi}^*$ that maximizes $V^{\boldsymbol{\pi}}(s_0)$ for every initial state s_0 (or, in the finite-horizon case, for a given initial state distribution).

Theorem 3.5 (NEXP-Completeness of Dec-POMDP)**.** *The problem of determining whether there exists a joint policy for a finite-horizon Dec-POMDP whose expected total reward exceeds a given threshold k is NEXP-complete.*

Proof sketch. We outline the two directions following Bernstein et al. [2002].

Membership in NEXP. A nondeterministic exponential-time algorithm can guess a joint policy (which, for a finite horizon T, is specified by at most N decision trees each of depth T, hence of exponential size) and verify it in exponential time by computing the expected reward via dynamic programming over the joint belief space.

NEXP-hardness. The reduction proceeds from the NEXP-complete *tiling problem*: given a finite set of tile types with matching constraints on edges, determine whether an exponentially large grid $2^n \times 2^n$ can be validly tiled. The construction encodes the grid into a Dec-POMDP with two agents. Agent 1 is responsible for rows and agent 2 for columns. The global state encodes the current grid position (i, j). Each agent observes only its respective coordinate (i or j) and must output a tile type for that position. The reward function assigns $+1$ if the chosen tile is consistent with all adjacency constraints for position (i, j) and 0 otherwise. The key difficulty is that the agents cannot communicate their tile choices to each other, so a valid tiling can be achieved if and only if there exist local policies (deterministic history-dependent strategies) that jointly produce a valid tiling. Since the grid has $2^n \times 2^n$ cells, the observation histories have exponential length, matching the NEXP complexity.

For the complete formal construction, see Bernstein et al. [2002, Theorem 3.1]. □

Computational Intractability of Dec-POMDPs

Finding an optimal joint policy for a Dec-POMDP is NEXP-complete (Theorem 3.5). This stands in contrast to:

- Single-agent MDPs: solvable in polynomial time (P) via linear programming [Puterman, 2014].
- Single-agent POMDPs: PSPACE-complete.
- Centralized multi-agent MDPs (full observability, common reward): solvable in P as a single large MDP.

The source of intractability is the combination of *decentralization* (local policies) and *partial observability* (agents cannot condition on global state).

Example 3.6 (Two-Agent Dec-POMDP: Cooperative Tiger)**.** Consider a variant of the classic tiger problem. Two agents stand before two doors. Behind one door is a tiger (penalty -50);

behind the other is a reward (+10). Each agent independently listens (receiving a noisy signal about the tiger's location with accuracy 0.85) or opens a door. If either agent opens the tiger door, both receive −50; if either opens the reward door (and neither opens the tiger door), both receive +10; if both listen, both receive −1.

Even in this two-agent, two-state, three-action problem, the optimal joint policy requires agents to coordinate the number of listening steps before committing. Agent 1 cannot condition on agent 2's observations, so the optimal policy must balance information gathering against the risk of uncoordinated door opening. Solving this optimally requires enumerating all possible joint observation histories—an exponential computation even for moderate horizons.

We now turn to practical frameworks that sidestep the NEXP barrier by exploiting additional structure available during training.

4 Centralized Training with Decentralized Execution

The NEXP-completeness of Dec-POMDPs (Theorem 3.5) motivates approximation frameworks that exploit asymmetry between training and execution phases.

Definition 4.1 (CTDE Paradigm)**.** The *Centralized Training with Decentralized Execution* (CTDE) paradigm is a two-phase approach to cooperative MARL:

1. **Training phase.** A centralized learning algorithm has access to global information: the state s_t, all agents' actions $\mathbf{a}_t = (a_1^t, \ldots, a_N^t)$, all observations $\mathbf{o}_t = (o_1^t, \ldots, o_N^t)$, and the shared reward $r_t = R(s_t, \mathbf{a}_t)$.
2. **Execution phase.** Each agent i executes a *decentralized policy* $\pi_i(\cdot \mid \tau_i)$ that conditions only on its local observation-action history τ_i.

The CTDE framework does *not* change the Dec-POMDP model or its complexity class. Rather, it relaxes the *learning* problem: during training, we may use global information to shape the learning signal (e.g., a centralized critic), provided the resulting policies are decentralizable.

Centralized critic, decentralized actors. A natural instantiation of CTDE uses a *centralized value function* $V(s)$ or $Q(s, \mathbf{a})$ that conditions on the global state (or joint observations) to provide low-variance gradient estimates. Each agent i maintains a local actor $\pi_i(\cdot \mid \tau_i)$ that is updated using policy gradient or value-based methods with the centralized critic as the learning signal.

CTDE as the Dominant Paradigm

Virtually all modern cooperative MARL algorithms adopt the CTDE paradigm, including:

- **Value decomposition** (VDN, QMIX, QTRAN): train a centralized Q_{tot} that factorizes into per-agent utilities.
- **Centralized critic methods** (MAPPO, MADDPG): train a centralized value function to reduce variance of decentralized policy gradients.
- **Communication methods** (CommNet, TarMAC): train communication protocols that enable richer coordination while keeping execution decentralized.

CTDE is successful because modern training environments (simulators) provide full state access, while real-world deployment demands decentralization due to communication constraints.

The remainder of this lesson develops two families of CTDE algorithms: value decomposition methods (Sections 5–7) and policy gradient methods (Section 8).

5 Value Decomposition Networks (VDN)

The first approach to cooperative MARL under CTDE decomposes the joint action-value function into per-agent utilities, enabling decentralized greedy action selection.

Problem setup. Suppose the team operates in a multi-agent MDP (fully observable variant of Dec-POMDP) or that each agent maintains an observation history τ_i. We seek a joint action-value function $Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a})$ where $\boldsymbol{\tau} = (\tau_1, \ldots, \tau_N)$ and $\mathbf{a} = (a_1, \ldots, a_N)$. For decentralized execution, each agent i must be able to select a_i using only τ_i, without knowledge of other agents' histories or actions.

Definition 5.1 (Additive Value Decomposition)**.** The *Value Decomposition Network* (VDN) [Sunehag et al., 2018] decomposes the joint action-value function as an additive sum:

$$Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}) = \sum_{i=1}^{N} Q_i(\tau_i, a_i),$$

where each $Q_i : \mathcal{T}_i \times \mathcal{A}_i \to \mathbb{R}$ is agent i's individual utility function.

Theorem 5.2 (Decentralized Greedy Action Selection under VDN)**.** *Under the additive decomposition of Definition 5.1, the greedy joint action decomposes into individual greedy actions:*

$$\arg\max_{\mathbf{a} \in \boldsymbol{\mathcal{A}}} Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}) = \left(\arg\max_{a_1 \in \mathcal{A}_1} Q_1(\tau_1, a_1),\ \ldots,\ \arg\max_{a_N \in \mathcal{A}_N} Q_N(\tau_N, a_N)\right).$$

Proof. Since $Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}) = \sum_{i=1}^{N} Q_i(\tau_i, a_i)$, and each term $Q_i(\tau_i, a_i)$ depends only on a_i, the joint maximization decomposes into N independent maximizations:

$$\max_{\mathbf{a} \in \boldsymbol{\mathcal{A}}} \sum_{i=1}^{N} Q_i(\tau_i, a_i) = \sum_{i=1}^{N} \max_{a_i \in \mathcal{A}_i} Q_i(\tau_i, a_i).$$

This equality holds because the feasible set $\boldsymbol{\mathcal{A}} = \mathcal{A}_1 \times \cdots \times \mathcal{A}_N$ is a Cartesian product, and the objective is separable. Any joint action $\mathbf{a}^* = (a_1^*, \ldots, a_N^*)$ achieving the left-hand side must satisfy $a_i^* \in \arg\max_{a_i} Q_i(\tau_i, a_i)$ for each i, since otherwise replacing a_i^* with a maximizer of Q_i would strictly increase the sum. □

Example 5.3 (VDN in a Two-Agent Delivery Task)**.** Consider two robots ($N = 2$) delivering packages in a grid. Robot 1 operates in the left half and robot 2 in the right half, with minimal interaction. Let $Q_1(\tau_1, a_1)$ represent the expected packages robot 1 delivers, and similarly for Q_2. Since the robots operate in disjoint regions, the team reward is approximately $R \approx R_1 + R_2$, so the true Q_{tot} is well-approximated by an additive decomposition. Each robot independently maximizes its own Q_i, and the joint action is jointly optimal.

Example 5.4 (Failure of VDN: Non-Additive Coordination)**.** Consider a two-agent, two-action coordination game with the following joint reward matrix:

$R(a_1, a_2)$	$a_2 = L$	$a_2 = R$
$a_1 = L$	8	0
$a_1 = R$	0	5

The optimal joint action is (L, L) with reward 8. Suppose VDN decomposes as $Q_{\text{tot}}(a_1, a_2) = Q_1(a_1) + Q_2(a_2)$. Then we need $Q_1(L) + Q_2(L) = 8$ and $Q_1(R) + Q_2(R) = 5$, but also $Q_1(L) + Q_2(R) = 0$ and $Q_1(R) + Q_2(L) = 0$. From these four equations: $Q_1(L) - Q_1(R) = 8 - 0 = 8$ (first and fourth) but also $Q_1(L) - Q_1(R) = 0 - 5 = -5$ (third and second), a contradiction. Thus VDN *cannot* represent this game's payoff structure.

The failure in Example 5.4 arises because the reward exhibits *interaction effects* between agents' actions. This motivates richer decomposition classes.

6 QMIX and the IGM Condition

To overcome the representational limitations of VDN while preserving decentralized action selection, Rashid et al. [2018] introduced QMIX, which relies on the *Individual-Global-Max* (IGM) condition.

Definition 6.1 (Individual-Global-Max (IGM) Condition)**.** A factorization $(Q_{\text{tot}}, Q_1, \ldots, Q_N)$ satisfies the *Individual-Global-Max* (IGM) condition if for all $\boldsymbol{\tau}$:

$$\arg\max_{\mathbf{a}\in\boldsymbol{\mathcal{A}}} Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}) = \left(\arg\max_{a_1\in\mathcal{A}_1} Q_1(\tau_1, a_1),\ \ldots,\ \arg\max_{a_N\in\mathcal{A}_N} Q_N(\tau_N, a_N)\right).$$

That is, the globally optimal joint action can be recovered by each agent independently maximizing its own utility Q_i.

IGM is the minimal structural condition needed for decentralized execution: if it holds, each agent can compute its part of the optimal joint action using only local information. The challenge is to design Q_{tot} as a function of $Q_1, \ldots, Q_N$ such that IGM is guaranteed.

Definition 6.2 (Monotonicity Condition)**.** A mixing function $Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}) = f(Q_1(\tau_1, a_1), \ldots, Q_N(\tau_N, a_N); \boldsymbol{\tau})$ satisfies the *monotonicity condition* if

$$\frac{\partial Q_{\text{tot}}}{\partial Q_i} \geq 0 \quad \text{for all } i \in \mathcal{N}.$$

Proposition 6.3 (Monotonicity Implies IGM)**.** *If $Q_{\text{tot}} = f(Q_1, \ldots, Q_N; \boldsymbol{\tau})$ satisfies the monotonicity condition (Definition 6.2), then the IGM condition (Definition 6.1) holds.*

Proof. Let $\bar{a}_i \in \arg\max_{a_i\in\mathcal{A}_i} Q_i(\tau_i, a_i)$ for each i, and let $\bar{\mathbf{a}} = (\bar{a}_1, \ldots, \bar{a}_N)$. We must show that $\bar{\mathbf{a}} \in \arg\max_{\mathbf{a}} Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a})$.

Take any $\mathbf{a} = (a_1, \ldots, a_N) \in \boldsymbol{\mathcal{A}}$. We construct a sequence of joint actions interpolating from $\mathbf{a}$ to $\bar{\mathbf{a}}$ by replacing one component at a time. Define $\mathbf{a}^{(0)} = \mathbf{a}$ and for $k = 1, \ldots, N$:

$$\mathbf{a}^{(k)} = (\bar{a}_1, \ldots, \bar{a}_k, a_{k+1}, \ldots, a_N).$$

So $\mathbf{a}^{(N)} = \bar{\mathbf{a}}$. Consider the transition from $\mathbf{a}^{(k-1)}$ to $\mathbf{a}^{(k)}$, which replaces a_k with $\bar{a}_k$. Since $\bar{a}_k$ maximizes $Q_k(\tau_k, \cdot)$, we have $Q_k(\tau_k, \bar{a}_k) \geq Q_k(\tau_k, a_k)$. The monotonicity condition ensures Q_{tot} is non-decreasing in each Q_i. Therefore, increasing (or not changing) Q_k while holding all other Q_j fixed cannot decrease Q_{tot}:

$$Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}^{(k)}) = f(\ldots, Q_k(\tau_k, \bar{a}_k), \ldots) \geq f(\ldots, Q_k(\tau_k, a_k), \ldots) = Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}^{(k-1)}).$$

Chaining over $k = 1, \ldots, N$:

$$Q_{\text{tot}}(\boldsymbol{\tau}, \bar{\mathbf{a}}) = Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}^{(N)}) \geq Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}^{(0)}) = Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}).$$

Since $\mathbf{a}$ was arbitrary, $\bar{\mathbf{a}}$ maximizes Q_{tot}. □

QMIX architecture. QMIX [Rashid et al., 2018] instantiates the monotonicity condition using a *mixing network*. Each agent i maintains a utility network $Q_i(\tau_i, a_i; \phi_i)$ parameterized by ϕ_i. The mixing network computes:

$$Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}; s) = f_{\text{mix}}\big(Q_1(\tau_1, a_1), \ldots, Q_N(\tau_N, a_N);\ s\big), \tag{1}$$

where f_{mix} is a feedforward network whose weights are generated by *hypernetworks* that take the global state s as input. The critical architectural choice is that the hypernetworks produce mixing weights via an absolute value activation:

$$W_\ell(s) = |g_\ell(s)|,$$

where g_ℓ is a learnable function of s and $|\cdot|$ denotes element-wise absolute value. This ensures $W_\ell(s) \geq 0$, so each layer of the mixing network is a non-negative linear combination (plus bias) followed by an ELU activation. Since compositions of non-decreasing functions are non-decreasing, the overall f_{mix} satisfies $\partial Q_{\text{tot}}/\partial Q_i \geq 0$.

Theorem 6.4 (QMIX Strictly Generalizes VDN)**.** *The class of Q_{tot} functions representable by QMIX strictly contains the class representable by VDN. That is, every VDN decomposition is a special case of QMIX, and there exist Q_{tot} functions representable by QMIX but not by VDN.*

Proof. VDN $\subseteq$ QMIX. The VDN decomposition $Q_{\text{tot}} = \sum_i Q_i$ is a special case of a monotonic mixing function: take $f_{\text{mix}}(q_1, \ldots, q_N) = \sum_i q_i$, which is a single-layer linear network with all weights equal to 1 and zero bias. This satisfies $\partial Q_{\text{tot}}/\partial Q_i = 1 > 0$.

Strict inclusion. Consider $N = 2$ agents with $Q_1, Q_2 \in \mathbb{R}$ and the mixing function $Q_{\text{tot}} = Q_1 \cdot Q_2$ (product, not sum). When $Q_1, Q_2 > 0$, we have $\partial Q_{\text{tot}}/\partial Q_i = Q_j > 0$, satisfying monotonicity. This is representable by a QMIX network (a two-layer mixing network can implement multiplication for positive inputs). However, $Q_{\text{tot}} = Q_1 \cdot Q_2$ is not additive: there exist no functions f, g such that $Q_1 \cdot Q_2 = f(Q_1) + g(Q_2)$ for all $Q_1, Q_2 > 0$. To verify, suppose $Q_1 Q_2 = f(Q_1) + g(Q_2)$. Setting $Q_2 = 1$: $Q_1 = f(Q_1) + g(1)$, so $f(Q_1) = Q_1 - c$ for $c = g(1)$. Setting $Q_1 = 1$: $Q_2 = (1 - c) + g(Q_2)$, so $g(Q_2) = Q_2 - 1 + c$. Then $f(Q_1) + g(Q_2) = Q_1 + Q_2 - 1 \neq Q_1 Q_2$ in general. □

IGM and Monotonic Value Factorization

- The **IGM condition** (Definition 6.1) is the necessary and sufficient structural property for decentralized greedy action selection to recover the jointly optimal action.
- The **monotonicity condition** (Definition 6.2) is a sufficient condition for IGM that is architecturally enforceable via non-negative mixing weights.
- QMIX enforces monotonicity through hypernetwork-generated non-negative weights, yielding a class strictly larger than VDN (Theorem 6.4).

Limitations of Monotonicity

The monotonicity condition prevents QMIX from representing *non-monotonic* interactions between agents. Consider a game where the optimal strategy requires agent 1 to take action L when agent 2 takes L, but action R when agent 2 takes R—and the joint Q-values are such that increasing Q_2 for action L would require decreasing Q_{tot} for some joint actions. Formally, any game matrix where $Q_{\text{tot}}(L, L) > Q_{\text{tot}}(R, L)$ but $Q_{\text{tot}}(L, R) < Q_{\text{tot}}(R, R)$ requires $\partial Q_{\text{tot}}/\partial Q_1$ to change sign, violating monotonicity.

Example 6.5 (QMIX on a Predator-Prey Task)**.** Consider $N = 3$ predators on a grid who must surround a prey to capture it. The team reward is $+10$ for capture and 0 otherwise. Each predator i observes a 5×5 local patch and maintains $Q_i(\tau_i, a_i)$ representing its utility for moving in direction $a_i \in \{N, S, E, W, \text{stay}\}$. The mixing network takes the global state s (all positions) and produces non-negative weights that combine the Q_i values. Since successful capture requires all predators to be adjacent to the prey, the reward is *super-additive*: the value of the team is greater than the sum of individual values. QMIX's state-dependent mixing weights can capture this—when the global state shows two predators already adjacent, the mixing network upweights the third predator's contribution, reflecting the increased marginal value of the final encirclement move.

7 QTRAN: Beyond Monotonic Decomposition

While QMIX achieves strong empirical results, its monotonicity constraint limits the class of representable joint value functions. Son et al. [2019] introduced QTRAN, which achieves IGM without requiring monotonicity.

Motivation. Consider a two-agent game where the optimal joint action is (L, L) but the off-diagonal entries of the reward matrix violate monotonicity (as in the warning above). QMIX cannot represent such games, but IGM may still hold if we allow a more flexible factorization.

Definition 7.1 (QTRAN Factorization)**.** The QTRAN factorization introduces three components:

1. Individual utility functions $Q_i(\tau_i, a_i)$ for each agent i.
2. A joint state-value function $V_{\text{jt}}(\boldsymbol{\tau})$.
3. Individual state-value functions $V_i(\tau_i)$ for each agent i.

The factorization posits that for the optimal joint action $\bar{\mathbf{a}}(\boldsymbol{\tau}) = (\bar{a}_1, \ldots, \bar{a}_N)$ where $\bar{a}_i = \arg\max_{a_i} Q_i(\tau_i, a_i)$:

$$Q_{\text{tot}}(\boldsymbol{\tau}, \bar{\mathbf{a}}) = \sum_{i=1}^{N} Q_i(\tau_i, \bar{a}_i) + V_{\text{jt}}(\boldsymbol{\tau}) - \sum_{i=1}^{N} V_i(\tau_i), \tag{2}$$

$$Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}) \leq \sum_{i=1}^{N} Q_i(\tau_i, a_i) + V_{\text{jt}}(\boldsymbol{\tau}) - \sum_{i=1}^{N} V_i(\tau_i), \quad \forall\, \mathbf{a} \neq \bar{\mathbf{a}}. \tag{3}$$

Proposition 7.2 (QTRAN Conditions Imply IGM)**.** *If conditions (2)–(3) hold, then the IGM condition (Definition 6.1) is satisfied.*

Proof. Let $\bar{\mathbf{a}}$ be the joint action with $\bar{a}_i = \arg\max_{a_i} Q_i(\tau_i, a_i)$ for each i. We must show $\bar{\mathbf{a}} \in \arg\max_{\mathbf{a}} Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a})$.

For any $\mathbf{a} \neq \bar{\mathbf{a}}$, using (2) and (3):

$$\begin{aligned} Q_{\text{tot}}(\boldsymbol{\tau}, \bar{\mathbf{a}}) &= \sum_{i=1}^{N} Q_i(\tau_i, \bar{a}_i) + V_{\text{jt}}(\boldsymbol{\tau}) - \sum_{i=1}^{N} V_i(\tau_i) \\ &\geq \sum_{i=1}^{N} Q_i(\tau_i, a_i) + V_{\text{jt}}(\boldsymbol{\tau}) - \sum_{i=1}^{N} V_i(\tau_i) \\ &\geq Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}). \end{aligned}$$

The first inequality uses $\bar{a}_i = \arg\max_{a_i} Q_i(\tau_i, a_i)$, so $\sum_i Q_i(\tau_i, \bar{a}_i) \geq \sum_i Q_i(\tau_i, a_i)$. The second inequality is condition (3) rearranged. Therefore $Q_{\text{tot}}(\boldsymbol{\tau}, \bar{\mathbf{a}}) \geq Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a})$ for all $\mathbf{a}$, establishing IGM. □

Practical implementation. The QTRAN conditions (2)–(3) are difficult to enforce exactly. In practice, Son et al. [2019] relax them into soft constraints within the loss function. Define:

$$\hat{Q}_{\text{jt}}(\boldsymbol{\tau}, \mathbf{a}) = \sum_{i=1}^{N} Q_i(\tau_i, a_i) + V_{\text{jt}}(\boldsymbol{\tau}) - \sum_{i=1}^{N} V_i(\tau_i).$$

The QTRAN loss combines three terms:

$$\mathcal{L}_{\text{td}} = \left(Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}) - y_{\text{td}}\right)^2, \tag{4}$$

$$\mathcal{L}_{\text{opt}} = \left(\hat{Q}_{\text{jt}}(\boldsymbol{\tau}, \bar{\mathbf{a}}) - Q_{\text{tot}}(\boldsymbol{\tau}, \bar{\mathbf{a}})\right)^2, \tag{5}$$

$$\mathcal{L}_{\text{nopt}} = \left(\min\left(\hat{Q}_{\text{jt}}(\boldsymbol{\tau}, \mathbf{a}) - Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}),\, 0\right)\right)^2, \tag{6}$$

where y_{td} is the TD target. The loss $\mathcal{L}_{\text{opt}}$ enforces the equality condition (2), while $\mathcal{L}_{\text{nopt}}$ enforces the inequality (3) via a hinge-like penalty.

Representational hierarchy. We can now summarize the relationship among the three factorization methods.

Proposition 7.3 (Representational Ordering)**.** *In terms of the classes of joint value functions that satisfy IGM under each method:*

$$\mathcal{F}_{\text{VDN}} \subsetneq \mathcal{F}_{\text{QMIX}} \subsetneq \mathcal{F}_{\text{QTRAN}}.$$

Proof. $\mathcal{F}_{\text{VDN}} \subsetneq \mathcal{F}_{\text{QMIX}}$ was established in Theorem 6.4.

For $\mathcal{F}_{\text{QMIX}} \subseteq \mathcal{F}_{\text{QTRAN}}$: any monotonic factorization satisfies IGM (Proposition 6.3). For any such factorization, set $V_{\text{jt}}(\boldsymbol{\tau}) = Q_{\text{tot}}(\boldsymbol{\tau}, \bar{\mathbf{a}}) - \sum_i Q_i(\tau_i, \bar{a}_i) + \sum_i V_i(\tau_i)$ and choose $V_i(\tau_i) = \max_{a_i} Q_i(\tau_i, a_i)$. Then condition (2) holds by construction, and condition (3) holds because $\bar{\mathbf{a}}$ maximizes Q_{tot} under monotonicity.

For strict inclusion: Example 5.4 provides a game where VDN fails. We show QTRAN can represent it but QMIX cannot. The reward matrix has $R(L, L) = 8 > R(R, L) = 0$ but $R(L, R) = 0 < R(R, R) = 5$. Set $Q_1(L) = 1, Q_1(R) = 0$ (so $\bar{a}_1 = L$) and $Q_2(L) = 1, Q_2(R) = 0$ (so $\bar{a}_2 = L$). The individual maxima select (L, L), which is indeed optimal ($R(L, L) = 8$). Set $V_1(\tau_1) = 1, V_2(\tau_2) = 1$, and $V_{\text{jt}} = 8 - 1 - 1 + 1 + 1 = 8$. Then $\hat{Q}_{\text{jt}}(L, L) = 1 + 1 + 8 - 1 - 1 = 8 = R(L, L)$ (equality holds). For (L, R): $\hat{Q}_{\text{jt}}(L, R) = 1 + 0 + 8 - 1 - 1 = 7 \geq R(L, R) = 0$ (inequality holds). Similarly for other off-diagonal entries. However, QMIX requires monotonicity, which fails here since $R(L, L) > R(R, L)$ implies $Q_1(L) > Q_1(R)$ yet $R(L, R) < R(R, R)$ would require $Q_1(L) < Q_1(R)$ under any monotonic decomposition, a contradiction. □

8 Multi-Agent Proximal Policy Optimization

Value decomposition methods (VDN, QMIX, QTRAN) address cooperative MARL through value-based learning. An alternative CTDE approach uses *policy gradient* methods with a centralized critic. Multi-Agent PPO (MAPPO) [Yu et al., 2022] adapts the single-agent PPO algorithm to the multi-agent setting and has demonstrated surprisingly strong performance.

Setup. Each agent i maintains a decentralized stochastic policy $\pi_i(a_i \mid \tau_i; \theta_i)$ parameterized by θ_i. A centralized value function $V(s; \psi)$ parameterized by ψ estimates the expected return from global state s under the current joint policy.

Definition 8.1 (MAPPO Objective)**.** For each agent i, the MAPPO objective is the clipped surrogate:

$$L_i^{\text{CLIP}}(\theta_i) = \mathbb{E}_{\boldsymbol{\tau}, \mathbf{a}}\Big[\min\Big(r_i(\theta_i)\,\hat{A}_i,\ \text{clip}\big(r_i(\theta_i),\, 1 - \epsilon,\, 1 + \epsilon\big)\,\hat{A}_i\Big)\Big], \tag{7}$$

where:

- $r_i(\theta_i) = \pi_i(a_i \mid \tau_i; \theta_i) \,/\, \pi_i(a_i \mid \tau_i; \theta_i^{\text{old}})$ is the importance sampling ratio for agent i.
- $\hat{A}_i = \hat{A}(s_t, \mathbf{a}_t)$ is the advantage estimate computed from the centralized value function: $\hat{A}_i = \sum_{\ell=0}^{T-t-1} (\gamma\lambda)^\ell \delta_{t+\ell}$ with TD errors $\delta_t = r_t + \gamma\, V(s_{t+1}; \psi) - V(s_t; \psi)$ (GAE-λ).
- $\epsilon > 0$ is the clipping parameter.

Shared vs. independent parameters. In many cooperative tasks, agents are *homogeneous* (same observation and action spaces). MAPPO can exploit this with *parameter sharing*: all agents use the same policy network $\pi(a_i \mid \tau_i, i; \theta)$, where agent identity i is provided as an additional input. This dramatically reduces the number of parameters from $N \cdot |\theta|$ to $|\theta|$ and enables faster learning, though it restricts the class of representable joint policies to those with symmetric structure (up to the identity embedding).

Theorem 8.2 (Monotonic Improvement under MAPPO)**.** *Let $\boldsymbol{\pi} = (\pi_1, \ldots, \pi_N)$ be the current joint policy and $\boldsymbol{\pi}' = (\pi'_1, \ldots, \pi'_N)$ be the updated joint policy where each π'_i maximizes $L_i^{\text{CLIP}}(\theta_i)$. Under the following conditions:*

1. *The advantage estimates $\hat{A}_i$ are computed using the centralized value function $V(s; \psi)$ with GAE-λ.*
2. *The clipping parameter ϵ is sufficiently small.*
3. *Policy updates are performed sequentially (one agent at a time) or the total KL divergence $\sum_i \mathrm{KL}(\pi_i^{\text{old}} \| \pi'_i)$ is bounded.*

Then the joint policy performance satisfies

$$V^{\boldsymbol{\pi}'}(s) \geq V^{\boldsymbol{\pi}}(s) - C \sum_{i=1}^{N} \max_{\tau_i} \mathrm{KL}\big(\pi_i(\cdot \mid \tau_i; \theta_i^{\text{old}}) \,\big\|\, \pi_i(\cdot \mid \tau_i; \theta_i)\big), \tag{8}$$

where $C = \frac{4\gamma \max_{s,\mathbf{a}} |A^{\boldsymbol{\pi}}(s,\mathbf{a})|}{(1-\gamma)^2}$ and $A^{\boldsymbol{\pi}}$ is the joint advantage function.

Proof. The proof extends the single-agent PPO monotonic improvement bound [Yu et al., 2022, Theorem 1] to the multi-agent setting.

Define the joint advantage $A^{\boldsymbol{\pi}}(s, \mathbf{a}) = Q^{\boldsymbol{\pi}}(s, \mathbf{a}) - V^{\boldsymbol{\pi}}(s)$. By the performance difference lemma:

$$V^{\boldsymbol{\pi}'}(s) - V^{\boldsymbol{\pi}}(s) = \frac{1}{1-\gamma} \mathbb{E}_{s' \sim d^s_{\boldsymbol{\pi}'}} \left[\mathbb{E}_{\mathbf{a} \sim \boldsymbol{\pi}'(\cdot|s')} \big[A^{\boldsymbol{\pi}}(s', \mathbf{a}) \big] \right],$$

where $d^s_{\boldsymbol{\pi}'}$ is the discounted state visitation distribution under $\boldsymbol{\pi}'$ starting from s. The key step is bounding the difference in expectations under $\boldsymbol{\pi}'$ vs. $\boldsymbol{\pi}$. Since the joint policy factorizes as $\boldsymbol{\pi}'(\mathbf{a} \mid s) = \prod_i \pi'_i(a_i \mid \tau_i)$, the total variation distance between $\boldsymbol{\pi}'$ and $\boldsymbol{\pi}$ satisfies:

$$\|\boldsymbol{\pi}'(\cdot \mid s) - \boldsymbol{\pi}(\cdot \mid s)\|_{\text{TV}} \leq \sum_{i=1}^{N} \|\pi'_i(\cdot \mid \tau_i) - \pi_i(\cdot \mid \tau_i)\|_{\text{TV}}.$$

Applying Pinsker's inequality ($\|p - q\|_{\text{TV}} \leq \sqrt{\frac{1}{2} \mathrm{KL}(p \| q)}$) to each term and combining with the performance difference lemma yields the bound (8). □

MAPPO on SMAC

Yu et al. [2022] showed that MAPPO, despite its simplicity, matches or exceeds the performance of value decomposition methods (QMIX, QTRAN) on the StarCraft Multi-Agent Challenge. Key practical insights include:

- **Centralized value function:** using the global state s as input to V (rather than local observations) significantly reduces variance.
- **Parameter sharing:** homogeneous agents benefit from shared policy networks with agent-ID conditioning.
- **Large batch sizes:** PPO's on-policy nature requires large batches for stable multi-agent learning; typical batch sizes are 10–100× larger than single-agent settings.

9 Credit Assignment

A fundamental challenge in cooperative MARL is *credit assignment*: given a shared team reward, how should the contribution of each agent be assessed?

Definition 9.1 (Credit Assignment Problem)**.** The *credit assignment problem* in cooperative MARL is the problem of decomposing a team reward signal $R(s, \mathbf{a})$ into agent-specific feedback signals r_i such that optimizing each r_i individually leads to optimization of the team objective.

We survey three principled approaches.

9.1 Difference Rewards

Definition 9.2 (Difference Reward)**.** The *difference reward* for agent i is

$$D_i(s, \mathbf{a}) = R(s, \mathbf{a}) - R(s, (a_{-i}, c_i)), \tag{9}$$

where $c_i \in \mathcal{A}_i$ is a fixed *default action* for agent i, and (a_{-i}, c_i) denotes the joint action with agent i's action replaced by c_i.

Proposition 9.3 (Alignment of Difference Rewards)**.** *If each agent i maximizes the expected sum of discounted difference rewards $\mathbb{E}[\sum_t \gamma^t D_i(s_t, \mathbf{a}_t)]$, then the resulting joint policy is a Nash equilibrium of the common-payoff game with reward R.*

Proof. Fix agent i and suppose all other agents $j \neq i$ follow fixed policies π_{-i}. Agent i's expected return under difference rewards is:

$$J_i^D(\pi_i) = \mathbb{E}^{\pi_i, \pi_{-i}}\left[\sum_t \gamma^t D_i(s_t, \mathbf{a}_t)\right] = \mathbb{E}^{\pi_i, \pi_{-i}}\left[\sum_t \gamma^t R(s_t, \mathbf{a}_t)\right] - \mathbb{E}^{\pi_{-i}}\left[\sum_t \gamma^t R(s_t, (a_{-i}^t, c_i))\right].$$

The second term does not depend on π_i (since c_i is fixed and the other agents' policies are fixed). Therefore, $\arg\max_{\pi_i} J_i^D(\pi_i) = \arg\max_{\pi_i} \mathbb{E}^{\pi_i, \pi_{-i}}[\sum_t \gamma^t R(s_t, \mathbf{a}_t)]$. If this holds for all i simultaneously, each agent's policy is a best response to the others under the team reward R, which is the definition of Nash equilibrium. In a common-payoff game, Nash equilibria coincide with team-optimal policies when the equilibrium is Pareto-efficient. □

9.2 Shapley Value-Based Credit Assignment

The Shapley value, from cooperative game theory (Phase 01, Lesson 20), provides a principled way to attribute team performance to individual agents.

Definition 9.4 (Shapley Value for MARL)**.** Given a cooperative game with characteristic function $v : 2^{\mathcal{N}} \to \mathbb{R}$ where $v(S)$ represents the value achievable by coalition $S \subseteq \mathcal{N}$, the *Shapley value* of agent i is

$$\phi_i(v) = \sum_{S \subseteq \mathcal{N} \setminus \{i\}} \frac{|S|!\,(N - |S| - 1)!}{N!} \big(v(S \cup \{i\}) - v(S)\big). \tag{10}$$

In the MARL context, we define $v(S) = \mathbb{E}[R(s, \mathbf{a}_S, \mathbf{c}_{-S})]$, where agents in S follow their learned policies and agents outside S take default actions.

Remark 9.5. The Shapley value satisfies four desirable axioms: efficiency ($\sum_i \phi_i = v(\mathcal{N}) - v(\emptyset)$), symmetry, null player, and additivity. However, computing it exactly requires evaluating 2^N coalitions, making it intractable for large N. Approximations via sampling random permutations of agents are commonly used.

9.3 Counterfactual Baselines: COMA

Definition 9.6 (Counterfactual Advantage)**.** The *Counterfactual Multi-Agent* (COMA) advantage for agent i is

$$A_i^{\mathrm{CF}}(s, \mathbf{a}) = Q(s, \mathbf{a}) - \sum_{a_i' \in \mathcal{A}_i} \pi_i(a_i' \mid \tau_i)\, Q(s, (a_{-i}, a_i')), \tag{11}$$

where $Q(s, \mathbf{a})$ is the centralized joint action-value function and the baseline is the expected Q-value over agent i's policy, marginalizing out a_i while holding a_{-i} fixed.

Proposition 9.7 (Unbiasedness of COMA Baseline)**.** *The COMA advantage satisfies* $\mathbb{E}_{a_i\sim\pi_i}[A_i^{\mathrm{CF}}(s,\mathbf{a})] = 0$*, so it is a valid baseline for variance reduction in policy gradient methods.*

Proof.

$$\begin{aligned}\mathbb{E}_{a_i\sim\pi_i}\big[A_i^{\mathrm{CF}}(s,\mathbf{a})\big] &= \mathbb{E}_{a_i\sim\pi_i}\big[Q(s,\mathbf{a})\big] - \mathbb{E}_{a_i\sim\pi_i}\Big[\sum_{a_i'}\pi_i(a_i'\mid\tau_i)\,Q(s,(a_{-i},a_i'))\Big]\\ &= \sum_{a_i}\pi_i(a_i\mid\tau_i)\,Q(s,(a_{-i},a_i)) - \sum_{a_i'}\pi_i(a_i'\mid\tau_i)\,Q(s,(a_{-i},a_i'))\\ &= 0.\end{aligned}$$

The baseline is independent of the sampled a_i (it averages over all possible actions), so subtracting it does not introduce bias into the policy gradient. □

Credit Assignment in Practice

- **VDN/QMIX:** implicitly perform credit assignment through the learned individual utilities Q_i—each agent's Q_i reflects its "share" of the team value.
- **COMA:** explicitly computes counterfactual baselines, requiring a centralized critic that can evaluate $Q(s,(a_{-i},a_i'))$ for all a_i'.
- **Shapley values:** provide the most principled attribution but at exponential computational cost; practical only for small N or with sampling approximations.

10 Exercises

Exercise 10.1 (VDN Decomposition and IGM)**.** $(*)$
Let $N=3$ agents with action spaces $\mathcal{A}_i=\{L,R\}$ and a VDN decomposition $Q_{\mathrm{tot}}(\mathbf{a}) = Q_1(a_1)+Q_2(a_2)+Q_3(a_3)$ with $Q_1(L)=3, Q_1(R)=1$, $Q_2(L)=2, Q_2(R)=4$, $Q_3(L)=5, Q_3(R)=2$. Verify that the IGM condition holds by computing $\arg\max_{\mathbf{a}} Q_{\mathrm{tot}}(\mathbf{a})$ both jointly (by enumeration) and via individual maximization.

Exercise 10.2 (Joint Value in a Dec-POMDP)**.** $(*)$
Consider a 2-agent, 2-state Dec-POMDP with $\mathcal{S}=\{s_0,s_1\}$, $\mathcal{A}_i=\{L,R\}$, discount $\gamma=0.9$, and deterministic transitions: from s_0, joint action (L,L) leads to s_1 with reward 5; all other joint actions keep the state at s_0 with reward 1. From s_1, all actions lead to s_1 with reward 3. Assuming full observability (so each agent sees the state), compute $V^{\boldsymbol{\pi}}(s_0)$ for the joint policy $\boldsymbol{\pi}=(\text{always } L, \text{always } L)$.

Exercise 10.3 (Monotonicity Implies IGM – Formal Proof)**.** $(**)$
Let $f:\mathbb{R}^N\to\mathbb{R}$ be a continuously differentiable function satisfying $\frac{\partial f}{\partial x_i}\geq 0$ for all i and all $\mathbf{x}\in\mathbb{R}^N$. Let $g_i:\mathcal{A}_i\to\mathbb{R}$ for $i=1,\dots,N$, and define $Q_{\mathrm{tot}}(\mathbf{a}) = f(g_1(a_1),\dots,g_N(a_N))$. Prove that

$$\arg\max_{\mathbf{a}} Q_{\mathrm{tot}}(\mathbf{a}) \supseteq \big\{\big(\arg\max_{a_1} g_1(a_1),\dots,\arg\max_{a_N} g_N(a_N)\big)\big\}.$$

Under what additional condition on f is equality guaranteed?

Hint: Use the coordinate-wise replacement argument from Proposition 6.3. For equality, consider strict monotonicity.

Exercise 10.4 (QMIX Fails, QTRAN Succeeds)**.** $(**)$
Consider the 2-agent coordination game with reward matrix:

$R(a_1,a_2)$	$a_2=A$	$a_2=B$	$a_2=C$
$a_1=A$	8	0	6
$a_1=B$	0	8	0
$a_1=C$	6	0	6

(a) Show that no monotonic decomposition $Q_{\text{tot}}(a_1, a_2) = f(Q_1(a_1), Q_2(a_2))$ with f non-decreasing in both arguments can represent this game (so QMIX fails).

(b) Construct individual utilities Q_1, Q_2, joint value V_{jt}, and individual values V_1, V_2 such that the QTRAN conditions (2)–(3) hold with $\bar{\mathbf{a}} = (A, A)$.

Hint: For part (a), consider the implications of $R(A, B) = 0 < R(B, B) = 8$ and $R(A, A) = 8 > R(B, A) = 0$ for monotonicity in Q_1.

Exercise 10.5 (NEXP-Hardness and Exponential Policies)**.** $(***)$
In the proof sketch of Theorem 3.5, we noted that the reduction from the tiling problem encodes a $2^n \times 2^n$ grid.

(a) Explain why a Dec-POMDP with horizon $T = 2^{2n}$ requires policies that are, in general, exponentially large decision trees.

(b) Show that if agents could communicate one bit per time step, the complexity would drop from NEXP to NEXP $\cap$ co-NEXP (informally: communication reduces the "decentralization penalty"). Discuss what this implies for the value of communication protocols in cooperative MARL.

Exercise 10.6 (Credit Assignment Bias Analysis)**.** $(***)$
Consider N agents with a team reward $R(s, \mathbf{a}) = \sum_{i=1}^{N} f_i(s, a_i) + g(s, \mathbf{a})$, where f_i captures agent i's individual contribution and g captures interaction effects. Define the difference reward with default action c_i for each agent.

(a) Derive an expression for $D_i(s, \mathbf{a})$ in terms of f_i and g.

(b) Show that if $g \equiv 0$ (no interactions), then D_i equals $f_i(a_i) - f_i(c_i)$, so maximizing D_i is equivalent to maximizing f_i.

(c) When $g \not\equiv 0$, analyze the bias introduced by difference rewards. Specifically, show that the gradient $\nabla_{\theta_i}\mathbb{E}[D_i]$ may not align with $\nabla_{\theta_i}\mathbb{E}[R]$ and characterize the conditions under which they do align.

11 Summary

- The **Dec-POMDP** formalizes cooperative MARL as a tuple $(\mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}, \mathcal{P}, R, \{\Omega_i\}, O, \gamma)$ where all agents share a common reward R but observe only local information.
- Finding an optimal joint policy for a Dec-POMDP is **NEXP-complete** (Theorem 3.5), far harder than single-agent MDPs (P) or POMDPs (PSPACE-complete). The intractability stems from the combination of decentralization and partial observability.
- The **CTDE paradigm** (Centralized Training with Decentralized Execution) sidesteps the complexity barrier by using global information during training while producing decentralized policies for execution.
- **VDN** decomposes $Q_{\text{tot}} = \sum_i Q_i$, guaranteeing that greedy action selection decomposes, but cannot represent non-additive agent interactions (Theorem 5.2).
- The **IGM condition** (Definition 6.1) is the fundamental structural requirement for decentralized greedy action selection. The monotonicity condition $\partial Q_{\text{tot}}/\partial Q_i \geq 0$ is a sufficient condition for IGM (Proposition 6.3).
- **QMIX** enforces monotonicity via non-negative mixing network weights generated by hypernetworks, strictly generalizing VDN (Theorem 6.4).

- **QTRAN** achieves IGM without monotonicity by introducing an additive correction term and enforcing equality/inequality conditions (Proposition 7.2), yielding the representational hierarchy VDN $\subsetneq$ QMIX $\subsetneq$ QTRAN.
- **MAPPO** adapts PPO to multi-agent settings with a centralized value function, preserving approximate monotonic improvement guarantees (Theorem 8.2).
- **Credit assignment**—attributing team reward to individual agents—can be addressed via difference rewards, Shapley values, or counterfactual baselines (COMA), each offering different bias-variance-computation tradeoffs.

12 Further Reading

- **Competitive and mixed MARL (Lesson 63)** extends the multi-agent framework to zero-sum and general-sum games, covering Minimax-Q, Nash-Q, MADDPG, and self-play methods including PSRO.
- **Mean-field games (Lesson 64)** addresses the scalability challenge when $N \to \infty$ by replacing agent-level interactions with interactions against a population distribution.
- **Communication in MARL (Lesson 65)** studies learned communication protocols (CommNet, DIAL, TarMAC) that allow agents to share information during execution, relaxing the strict decentralization assumption of Dec-POMDPs.
- **QPLEX** [Wang et al., 2021] introduces duplex dueling architectures that decompose Q_{tot} into advantage and value components, achieving IGM with greater representational capacity than QMIX while maintaining tractable training.
- **MAVEN** [Mahajan et al., 2019] addresses the exploration challenge in value decomposition methods by introducing a shared latent variable that conditions agent exploration, promoting diverse joint behaviors.
- **SMAC benchmark** [Samvelyan et al., 2019] provides a standardized suite of StarCraft II micromanagement scenarios for evaluating cooperative MARL algorithms, with varying difficulty levels and team compositions.
- **Comprehensive surveys.** Zhang et al. [2021] provides a selective overview of MARL theory and algorithms. Oliehoek and Amato [2016, Chapters 6–8] covers approximate methods for Dec-POMDPs in depth. Kraemer and Banerjee [2016] surveys multi-agent reinforcement learning as a framework for decentralized planning.

References

Daniel S. Bernstein, Robert Givan, Neil Immerman, and Shlomo Zilberstein. The complexity of decentralized control of Markov decision processes. *Mathematics of Operations Research*, 27 (4):819–840, 2002.

Landon Kraemer and Bikramjit Banerjee. Multi-agent reinforcement learning as a rehearsal for decentralized planning. In *Neurocomputing*, volume 190, pages 82–94, 2016.

Anuj Mahajan, Tabish Rashid, Mikayel Samvelyan, and Shimon Whiteson. MAVEN: Multi-agent variational exploration. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

Frans A. Oliehoek and Christopher Amato. *A Concise Introduction to Decentralized POMDPs*. Springer, 2016.

Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 2nd edition, 2014.

Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 4295–4304, 2018.

Mikayel Samvelyan, Tabish Rashid, Christian Schroeder de Witt, Gregory Farquhar, Nantas Nardelli, Tim G. J. Rudner, Chia-Man Hung, Philip H. S. Torr, Jakob Foerster, and Shimon Whiteson. The StarCraft multi-agent challenge. In *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, pages 2186–2188, 2019.

Kyunghwan Son, Daewoo Kim, Wan Ju Kang, David Earl Hostallero, and Yung Yi. QTRAN: Learning to factorize with transformation for cooperative multi-agent reinforcement learning. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 5887–5896, 2019.

Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech Marian Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls, and Thore Graepel. Value-decomposition networks for cooperative multi-agent learning based on team reward. In *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, pages 2085–2087, 2018.

Jianhao Wang, Zhizhou Ren, Terry Liu, Yang Yu, and Ce Zhang. QPLEX: Duplex dueling multi-agent q-learning. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. *Advances in Neural Information Processing Systems (NeurIPS)*, 35:24611–24624, 2022.

Kaiqing Zhang, Zhuoran Yang, and Tamer Başar. Multi-agent reinforcement learning: A selective overview of theories and algorithms. *Handbook of Reinforcement Learning and Control*, pages 321–384, 2021.